Processes

What is a Process?

- Sequence of instructions that executes
  - A *program in execution*
  - The entity that can be assigned to and executed on a processor
  - A unit of activity characterized by a single sequential thread of execution
  - Can be traced: list the sequence of instructions that execute
- Associated data needed by the program
- Context
  - All information OS needs to manage the process
  - A current state and an associated set of system resources

File Options View

Processes | Performance | App history | Startup | Users | Details | Services

Name	Status	48% CPU	56% Memory	42% Disk	0% Network	0% GPU	GPU engine	Power usage	Power usage t...
Kaspersky Endpoint Security for...		42.6%	239.5 MB	273.8 MB/s	0.1 Mbps	0%		Very high	Moderate
System		1.0%	0.1 MB	1.4 MB/s	0 Mbps	0%		Very low	Very low
Task Manager		1.0%	32.0 MB	0 MB/s	0 Mbps	0%		Very low	Very low
System interrupts		0.6%	0 MB	0 MB/s	0 Mbps	0%		Very low	Very low
WMI Provider Host		0.5%	14.9 MB	0 MB/s	0 Mbps	0%		Very low	Very low
xagt.exe		0.5%	647.4 MB	0.1 MB/s	0 Mbps	0%		Very low	Very low
Cisco Webex Service		0.3%	4.0 MB	0 MB/s	0 Mbps	0%		Very low	Very low
HP Collaboration Keyboard		0.3%	6.2 MB	0 MB/s	0 Mbps	0%		Very low	Very low
xagt.exe		0.3%	161.7 MB	0 MB/s	0 Mbps	0%		Very low	Very low
Firefox (21)		0.2%	1,480.4 MB	0.1 MB/s	0 Mbps	0%		Very low	Very low
Service Host: Windows Manage...		0.2%	14.4 MB	0 MB/s	0 Mbps	0%		Very low	Very low
Desktop Window Manager		0.2%	137.4 MB	0 MB/s	0 Mbps	0.2%	GPU 0 - 3D	Very low	Very low
Microsoft Outlook (2)		0.1%	153.7 MB	0.1 MB/s	0 Mbps	0%		Very low	Very low
xagt.exe		0.1%	3.3 MB	0 MB/s	0 Mbps	0%		Very low	Very low
Client Server Runtime Process		0.1%	1.4 MB	0 MB/s	0 Mbps	0.1%	GPU 0 - 3D	Very low	Very low
Service Host: Remote Procedure...		0.1%	9.8 MB	0 MB/s	0 Mbps	0%		Very low	Very low
Service Host: Local Service (Net...		0.1%	2.3 MB	0 MB/s	0 Mbps	0%		Very low	Very low
Dropbox		0.1%	110.3 MB	0 MB/s	0 Mbps	0%		Very low	Very low

Task Manager

File Options View

Processes | Performance | App history | Startup | Users | Details | Services

Name	PID	Status	Username	CPU	Memory (ac...	UAC virtualisati...
System Idle Process	0	Running	SYSTEM	71	8 K	
avp.exe	5160	Running	SYSTEM	24	233,012 K	Not allowed
System interrupts	-	Running	SYSTEM	01	0 K	
System	4	Running	SYSTEM	01	20 K	
atmgr.exe	14912	Running	nadly	01	4,116 K	Disabled
Taskmgr.exe	12836	Running	nadly	01	31,420 K	Not allowed
msedge.exe	4840	Running	nadly	00	28,464 K	Disabled
firefox.exe	11852	Running	nadly	00	172,068 K	Disabled
svchost.exe	1468	Running	NETWORK ...	00	2,688 K	Not allowed
csrss.exe	3776	Running	SYSTEM	00	1,396 K	Not allowed
xagt.exe	9748	Running	SYSTEM	00	662,964 K	Not allowed
firefox.exe	12492	Running	nadly	00	400,176 K	Disabled
Veeam.EndPoint.Servi...	7556	Running	SYSTEM	00	18,448 K	Not allowed
dwm.exe	15380	Running	DWM-2	00	129,708 K	Disabled
HPCollaborationKeyb...	20200	Running	nadly	00	6,428 K	Not allowed
svchost.exe	4224	Running	LOCAL SER...	00	2,372 K	Not allowed
svchost.exe	5276	Running	LOCAL SER...	00	29,036 K	Not allowed
firefox.exe	17972	Running	nadly	00	280,948 K	Disabled
explorer.exe	1156	Running	nadly	00	62,508 K	Disabled
svchost.exe	11104	Running	SYSTEM	00	4,320 K	Not allowed
svchost.exe	2008	Running	SYSTEM	00	1,480 K	Not allowed
WmiPrvSE.exe	3584	Running	NETWORK ...	00	15,192 K	Not allowed
firefox.exe	13344	Running	nadly	00	150,108 K	Disabled
mDNSResponder.exe	5152	Running	SYSTEM	00	360 K	Not allowed
xagt.exe	9548	Running	SYSTEM	00	165,576 K	Not allowed
svchost.exe	2796	Running	LOCAL SER...	00	1,104 K	Not allowed
Dropbox.exe	14472	Running	nadly	00	15,804 K	Disabled

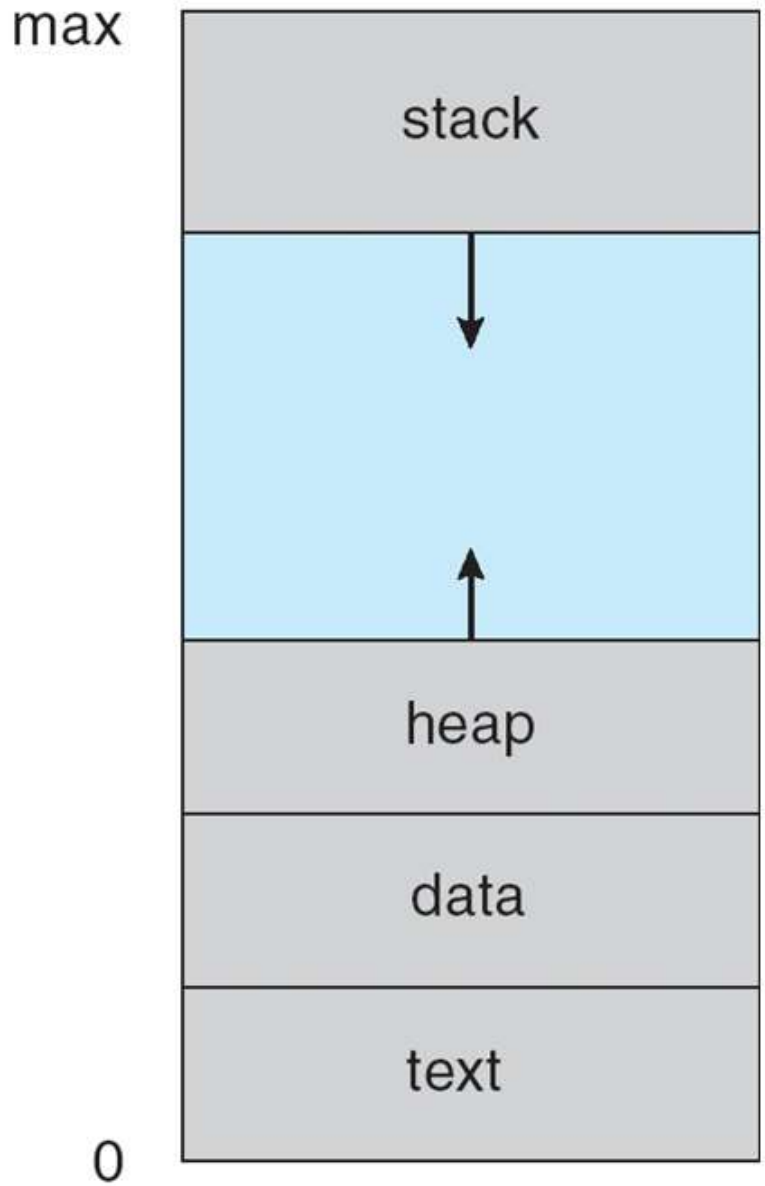
Requirements of an OS

- Interleave the execution of several processes to maximize processor utilization while providing reasonable response time
- Allocate resources to processes
- Protect the resources of each process from other processes
- Enable processes to share and exchange information; Support InterProcess Communication (IPC)
- Enable synchronization among processes
- Creation and termination of processes

All requirements are expressed with reference to **processes**

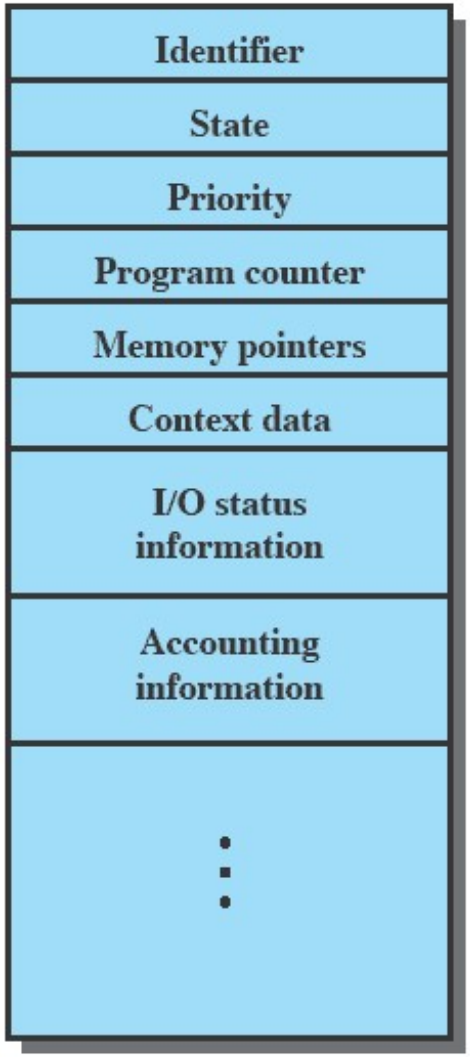
Process in Memory

- A Process consists of Multiple parts
  - The **program code**, also called text section
  - **Data section** containing global variables
  - Current **State** including program counter, processor registers
  - **Stack** containing temporary data
    - Function parameters, return addresses, local variables
  - **Heap** containing memory dynamically allocated during run time



Process Elements

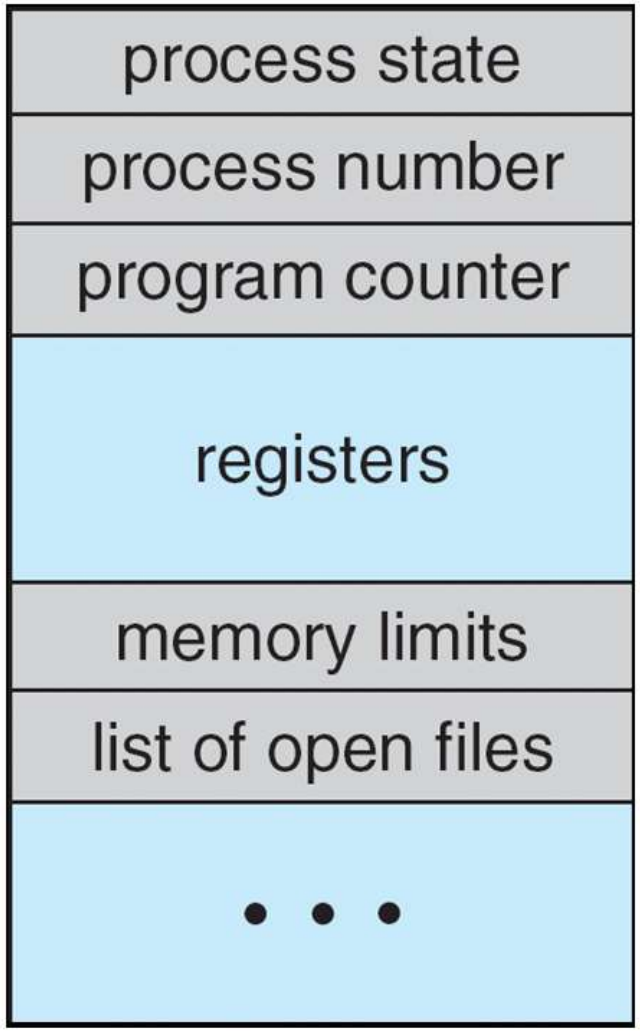
- Program code
- A set of data
- **Process Control Block**
  - Contains the process elements uniquely identifying the process
  - Created and managed by the OS
  - contains sufficient information that allow to interrupt a running process and later resume execution
  - Allows OS to support multiple processes



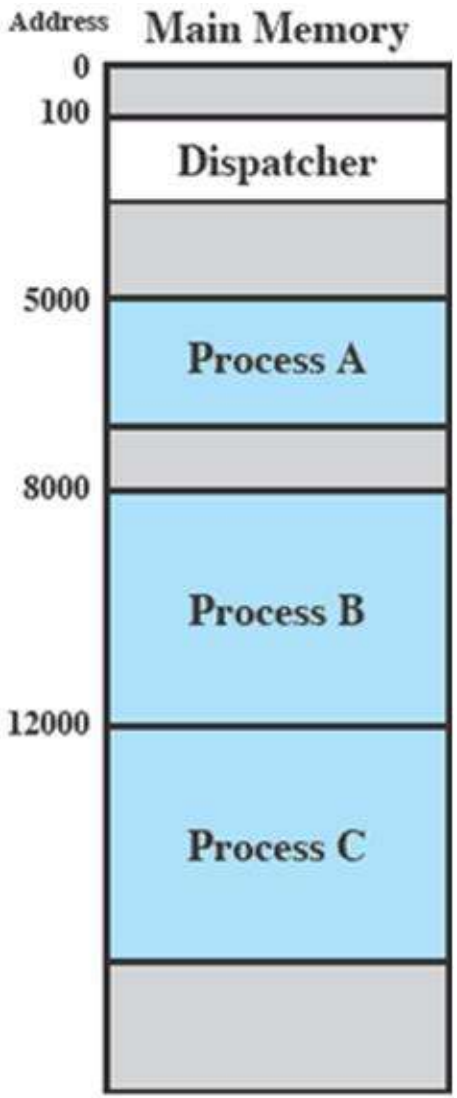
Process Control Block (PCB)

Information associated with each process

- Process state – running, waiting, etc
- Program counter – location of instruction to next execute
- CPU registers – contents of all process-centric registers
- CPU scheduling information- priorities, scheduling queue pointers
- Memory-management information – memory allocated to the process
- Accounting information – CPU used, clock time elapsed since start, time limits
- I/O status information – I/O devices allocated to process, list of open files

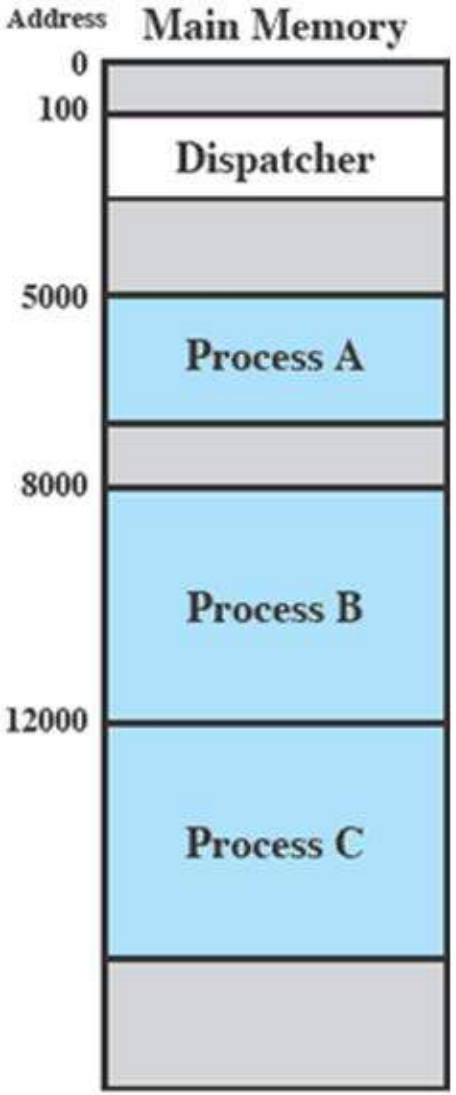


Process Execution



- Consider three processes being executed
- **Dispatcher** is a small program which switches the processor from one process to another
- All are in memory (plus the dispatcher)

Trace of Processes

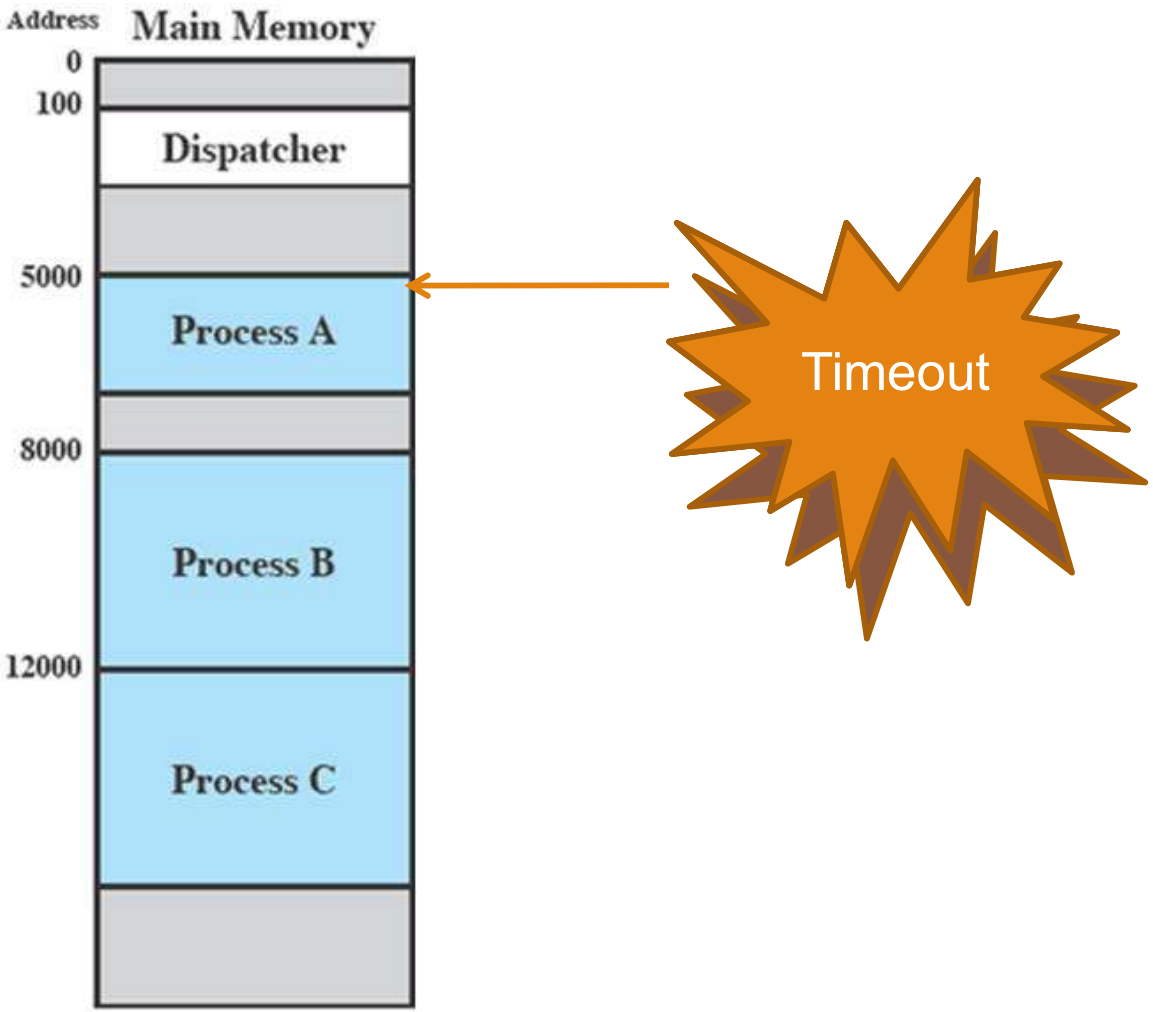


(a) Trace of Process A	(b) Trace of Process B	(c) Trace of Process C
5000	8000	12000
5001	8001	12001
5002	8002	12002
5003	8003	12003
5004		12004
5005		12005
5006		12006
5007		12007
5008		12008
5009		12009
5010		12010
5011		12011

5000 = Starting address of program of Process A
8000 = Starting address of program of Process B
12000 = Starting address of program of Process C

Figure 3.3 Traces of Processes of Figure 3.2

Trace from Processors point of view



1	5000	
2	5001	
3	5002	
4	5003	
5	5004	
6	5005	
		Timeout
7	100	
8	101	
9	102	
10	103	
11	104	
12	105	
13	8000	
14	8001	
15	8002	
16	8003	
		I/O Request
17	100	
18	101	
19	102	
20	103	
21	104	
22	105	
23	12000	
24	12001	
25	12002	
26	12003	
27	12004	
28	12005	
		Timeout
29	100	
30	101	
31	102	
32	103	
33	104	
34	105	
35	5006	
36	5007	
37	5008	
38	5009	
39	5010	
40	5011	
		Timeout
41	100	
42	101	
43	102	
44	103	
45	104	
46	105	
47	12006	
48	12007	
49	12008	
50	12009	
51	12010	
52	12011	
		Timeout

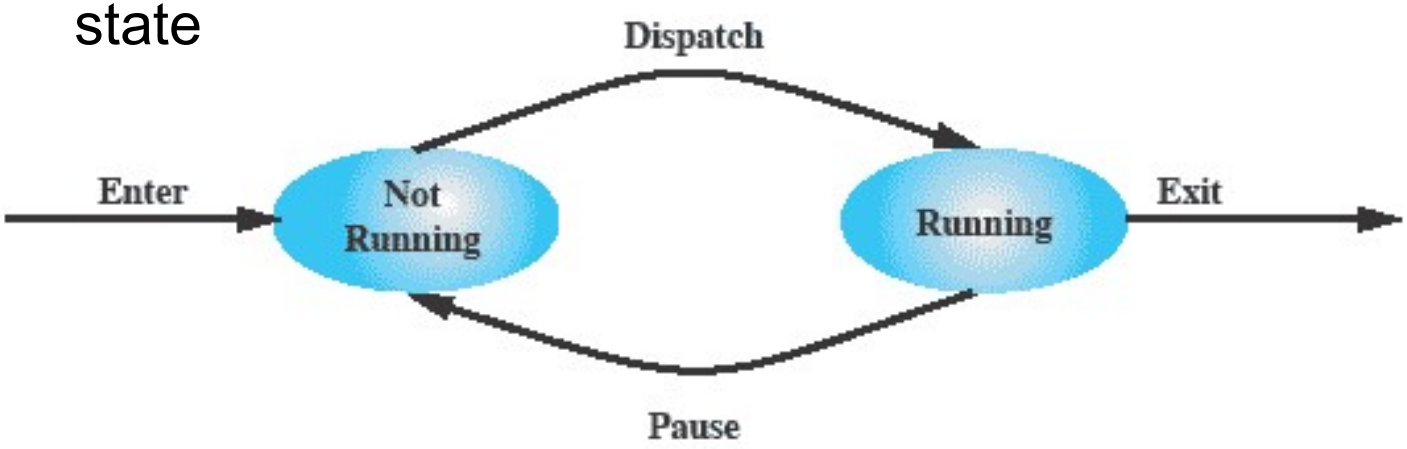
100 = Starting address of dispatcher program

Shaded areas indicate execution of dispatcher process;
first and third columns count instruction cycles;
second and fourth columns show address of instruction being executed

Figure 3.4 Combined Trace of Processes of Figure 3.2

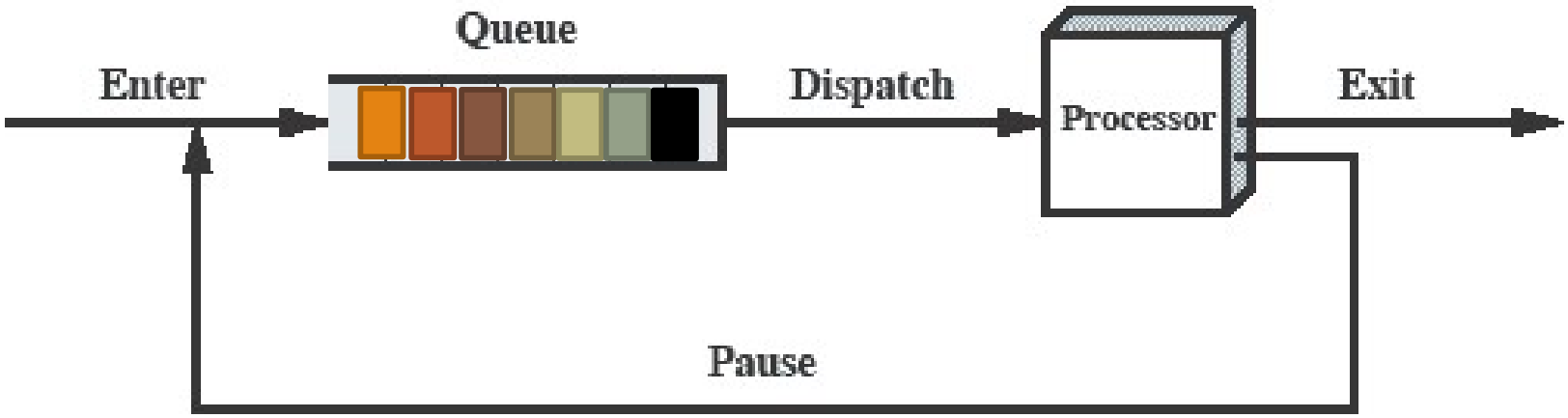
Two-State Process Model

- Process may be in one of two states
  - Running
  - Not-running
- When OS creates a new process
  - it creates a new Process Control Block PCB
  - When Dispatcher selects a process to run
    - Move Running process to Not Running state
    - Move selected process from Not Running state to Running state



Queuing Diagram

- Each process must be represented in a structure so; OS can keep track of it
- Could be a simple queue, managed by the dispatcher of the OS
- Each entry of Queue is a pointer to PCB of a process



Etc … processes moved by the dispatcher of the OS to the CPU then back to the queue until the task is competed

Life of a process is bound by its creation and termination

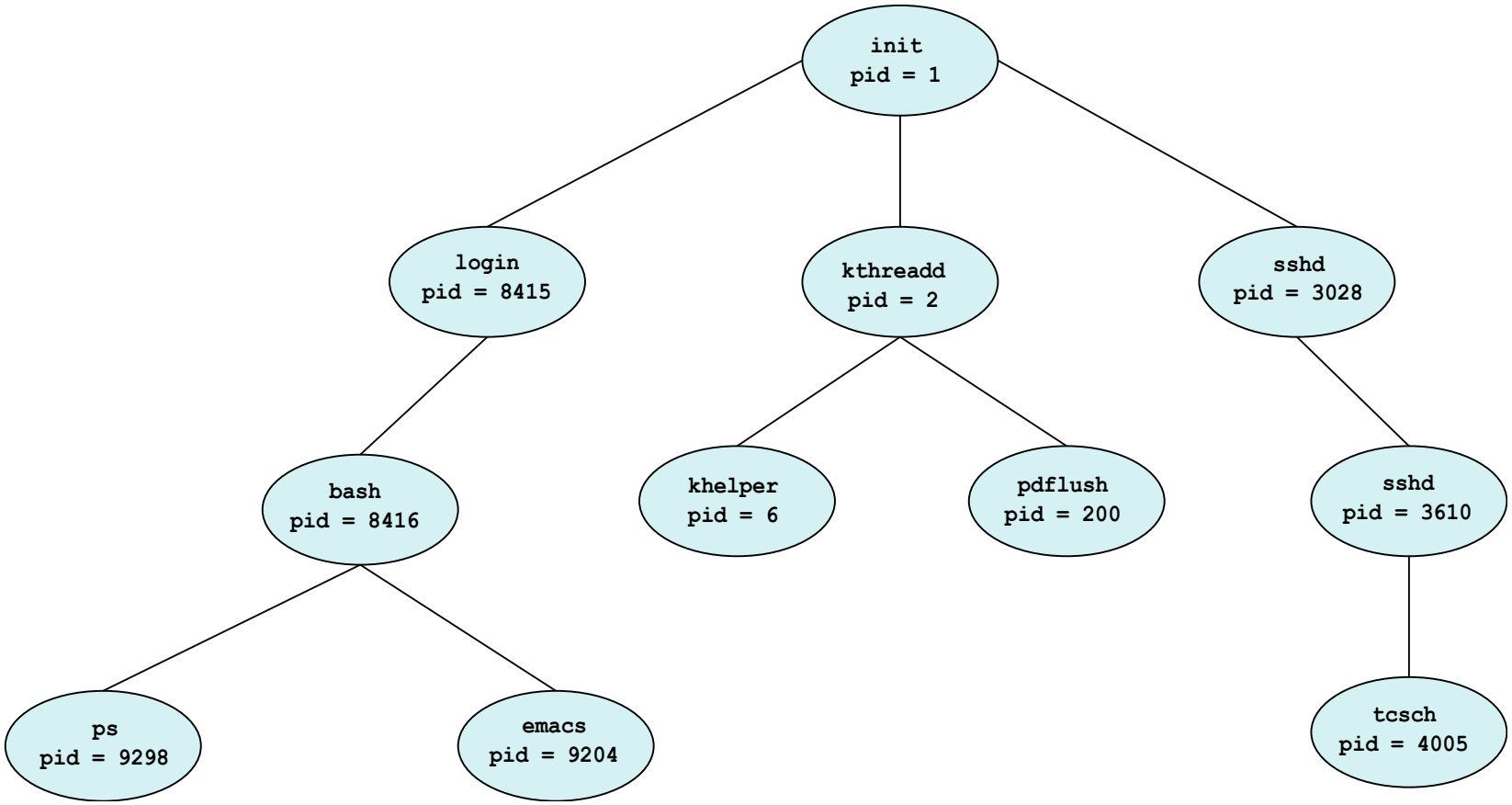
What Initiates Process Creation?

- Submission of a batch job
- User logs on
- Created to provide a service e.g. printing
- Process creates another process process spawning
  - Parent Process is the original, creating process
  - Child Process is the new process
  - Ex: file server generates a process for each request
  - Useful in structuring applications: Modularity
  - Parent and Child processes can run simultaneously: Parallelism

Deciding how to allocate the resources is a policy that is determined by the OS

Process Creation

- A process may create new processes: **Parent** process creates **children** processes, which, in turn create other processes, forming a **tree** of processes
- Process is identified and managed via a **process identifier (pid)**



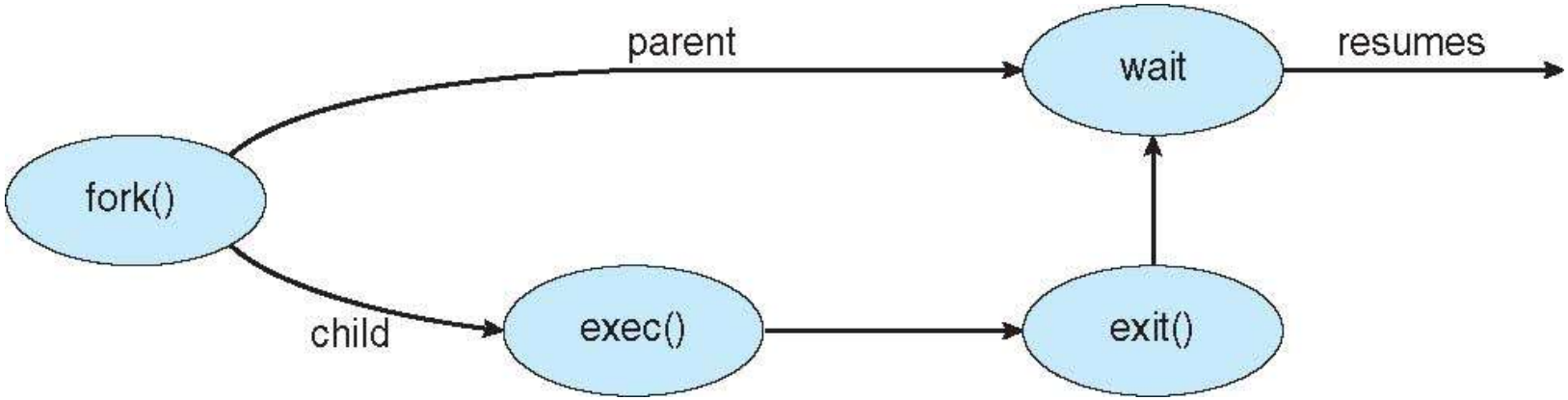
A Tree of Processes in Linux

Process Creation

- Resource sharing options
  - Parent and children share all resources
  - Children share subset of parent's resources
  - Parent and child share no resources
- Execution options
  - Parent and children execute concurrently
  - Parent waits until children terminate
- Address space
  - Child duplicate of parent
  - Child has a program loaded into it

Ex: UNIX Forking Separate Process

- UNIX examples
  - **`fork()`** system call creates new process
  - **`exec()`** system call used after a **`fork()`** to replace the process' memory space with a new program



C Program Forking Separate Process

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       return 1;
    }
    else if (pid == 0) { /* child process */
       execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
       /* parent will wait for the child to complete */
       wait(NULL);
       printf("Child Complete");
    }

    return 0;
}
```

Process Termination

- Process executes last statement and then asks the OS to delete it using the `exit()` system call.
  - Returns status data from child to parent (via `wait()`)
  - Process' resources are deallocated by operating system
- Parent may terminate the execution of children processes using the `abort()` system call. Some reasons for doing so:
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - The parent is exiting and the OS does not allow a child to continue if its parent terminates

Reasons for Process Termination

- User logs off
- Normal completion
- Batch issues *Halt*
- Time limit exceeded
- Memory unavailable
- Bounds violation
- Protection error
  - e.g. write to read-only file
- Arithmetic error
- Time overrun
  - event timeout
- I/O failure
- Invalid instruction
  - tried to execute data
- Privileged instruction
- Data misuse
- OS intervention
  - e.g. when deadlock occurs
- Parent terminates; so child processes terminate
- Parent request

Process Termination

- Some OS do not allow child to exist if its parent has terminated. If a process terminates, then all its children must also be terminated.
  - **cascading termination**. All children, grandchildren, etc. are terminated.
  - The termination is initiated by the OS.
- The parent process may wait for termination of a child process by using the **`wait()`** system call. The call returns status information and the pid of the terminated process

```
pid = wait(&status);
```

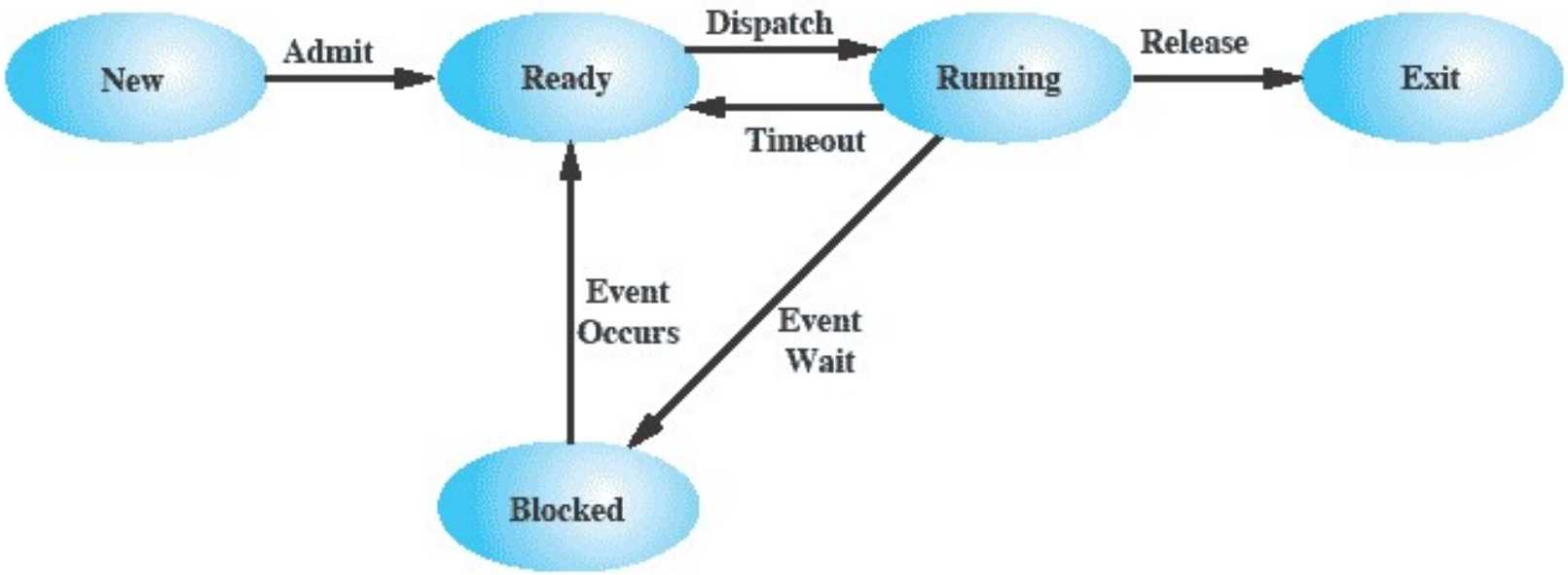
- If no parent waiting (did not invoke **`wait()`**) process is a **zombie**
- If parent terminated without invoking **`wait`**, process is an **orphan**

Multiprocess Architecture – Chrome Browser

- Many web browsers ran as single process (some still do)
  - If one web site causes trouble, entire browser can hang or crash
- Google Chrome Browser is multiprocess with 3 different types of processes:
  - **Browser** process manages user interface, disk and network I/O. Only one browser process is created per session
  - **Renderer** process renders web pages, deals with HTML, Javascript etc. A new renderer process is created for each website opened in a new tab
  - **Plug-in** process for each type of plug-in e.g. Flash, Quicktime etc..
  - **Adv:** websites runs un isolation from one another, if one crashes, only its renderer process is affected and other processes are unharmed.

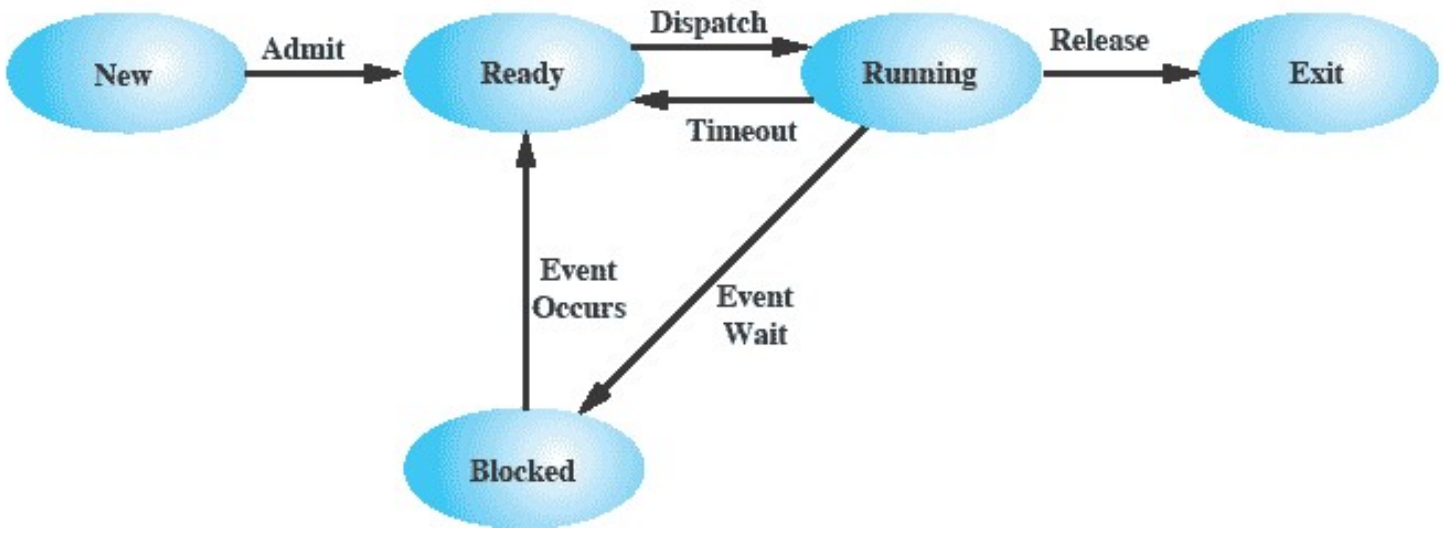
Five-State Process Model

- Blocked: waiting for I/O, semaphore – cannot run
- Using a single Queue, Dispatcher cannot just select the process that has been in the queue the longest because it may be blocked – has to scan the queue
- Split Not Running into Ready and Blocked

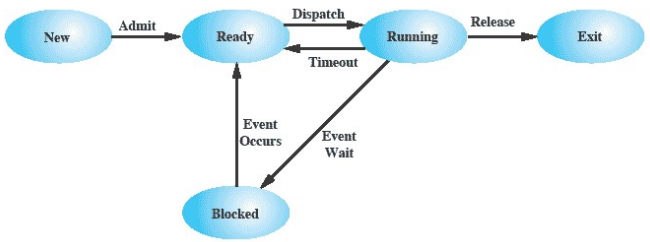


Five-State Process Model

Running: The process that is currently being executed.
Ready: A process ready to execute when given the opportunity
Blocked/Waiting: A process that cannot execute until some event occurs, such as the completion of an I/O operation
New: A process just created but not yet admitted to the pool of executable processes by the OS. (not yet loaded in main memory, although its PCB has been created)
Exit: A process released from the pool of executable processes



Five-state Model Transitions

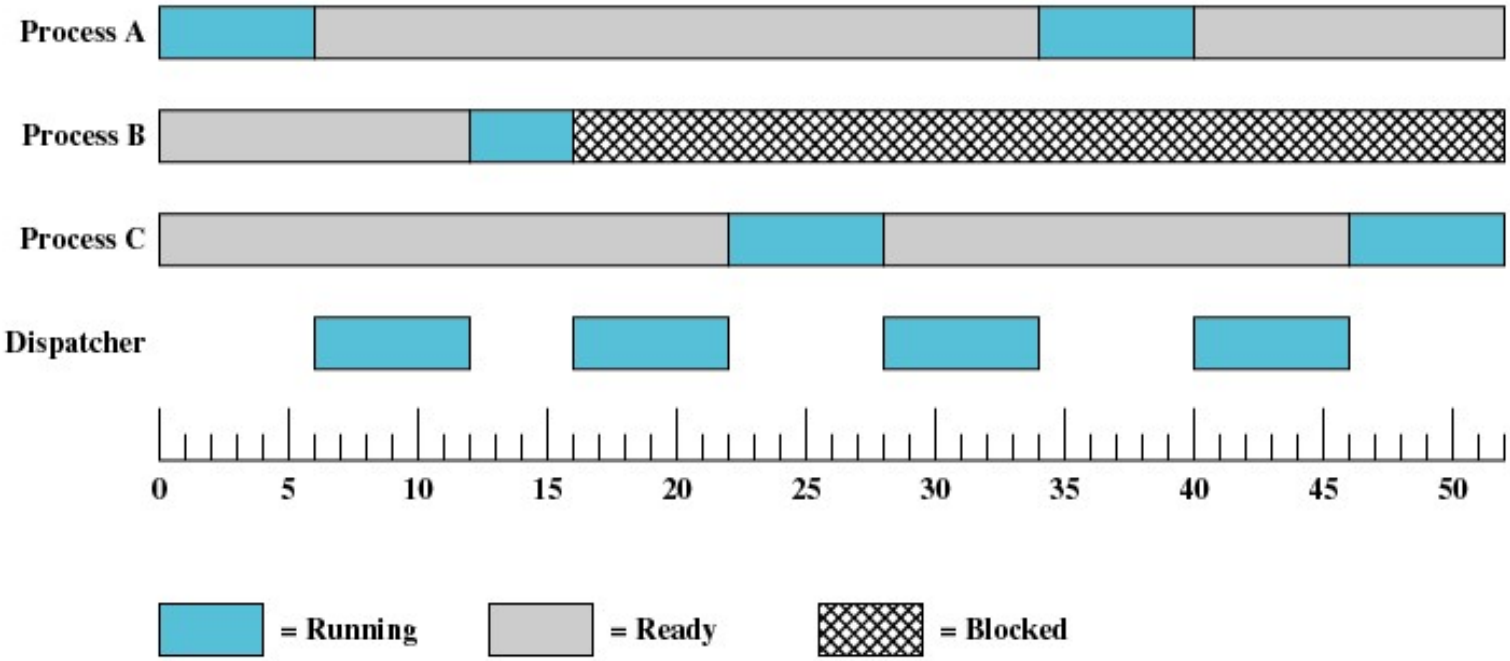


- **Null → New**: Process is created
- **New → Ready**: OS is ready to handle the process

(∃ limit on # processes for memory constraints)

- **Ready → Running**: Process is selected by OS to run
- **Running → Exit**: Process has terminated
- **Running → Ready**: End of time slice or higher-priority process is ready
- **Running → Blocked**: Process is waiting for an event
- **Blocked → Ready**: The event a process is waiting for has occurred, can continue
- **Ready → Exit**: Process terminated by O.S. or parent
- **Blocked → Exit**: Same reasons

Process States



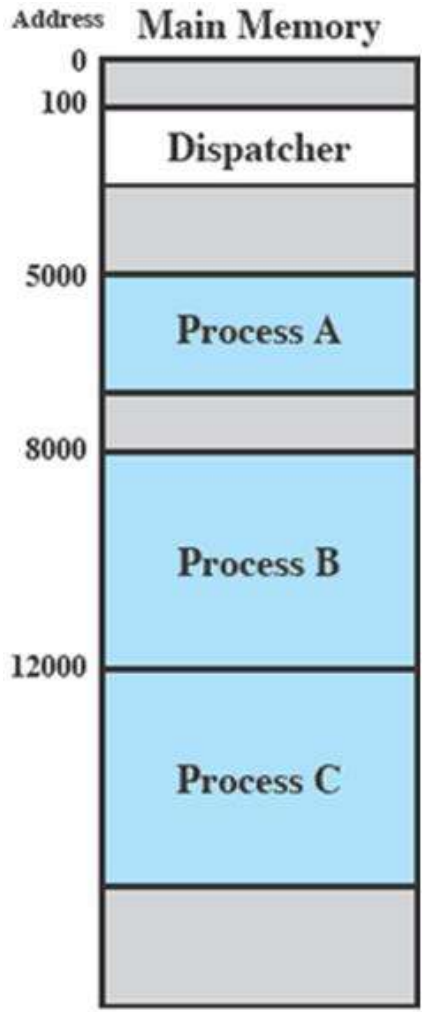
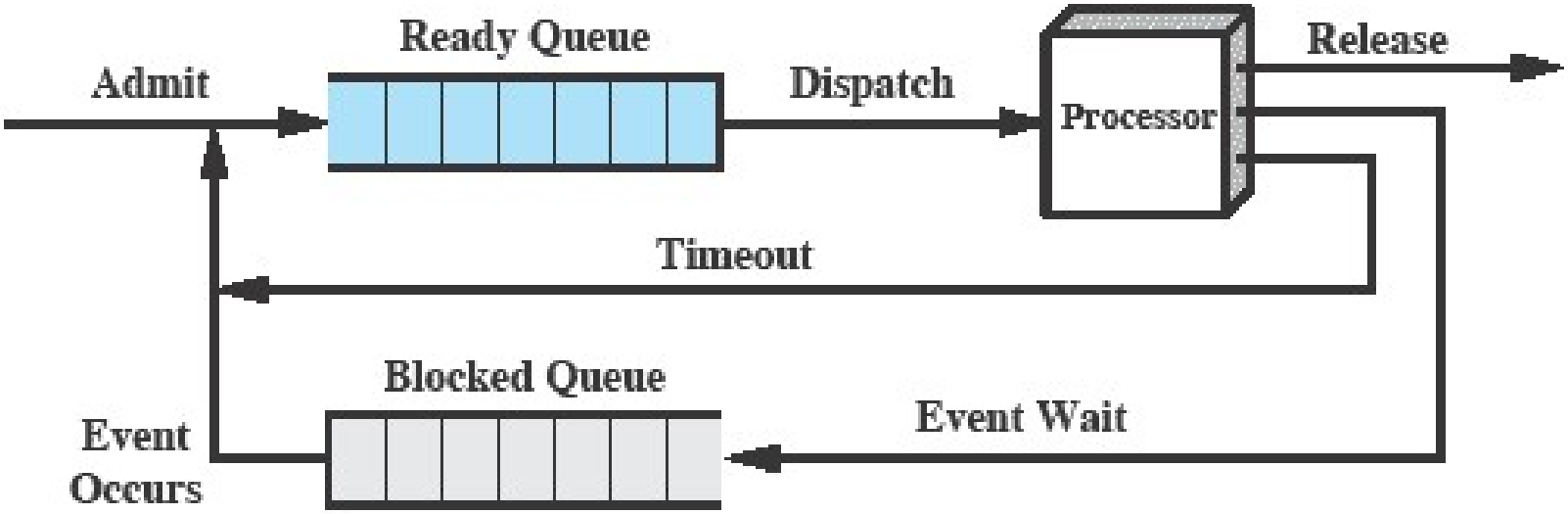


Figure 3.7 Process States for Trace of Figure 3.4

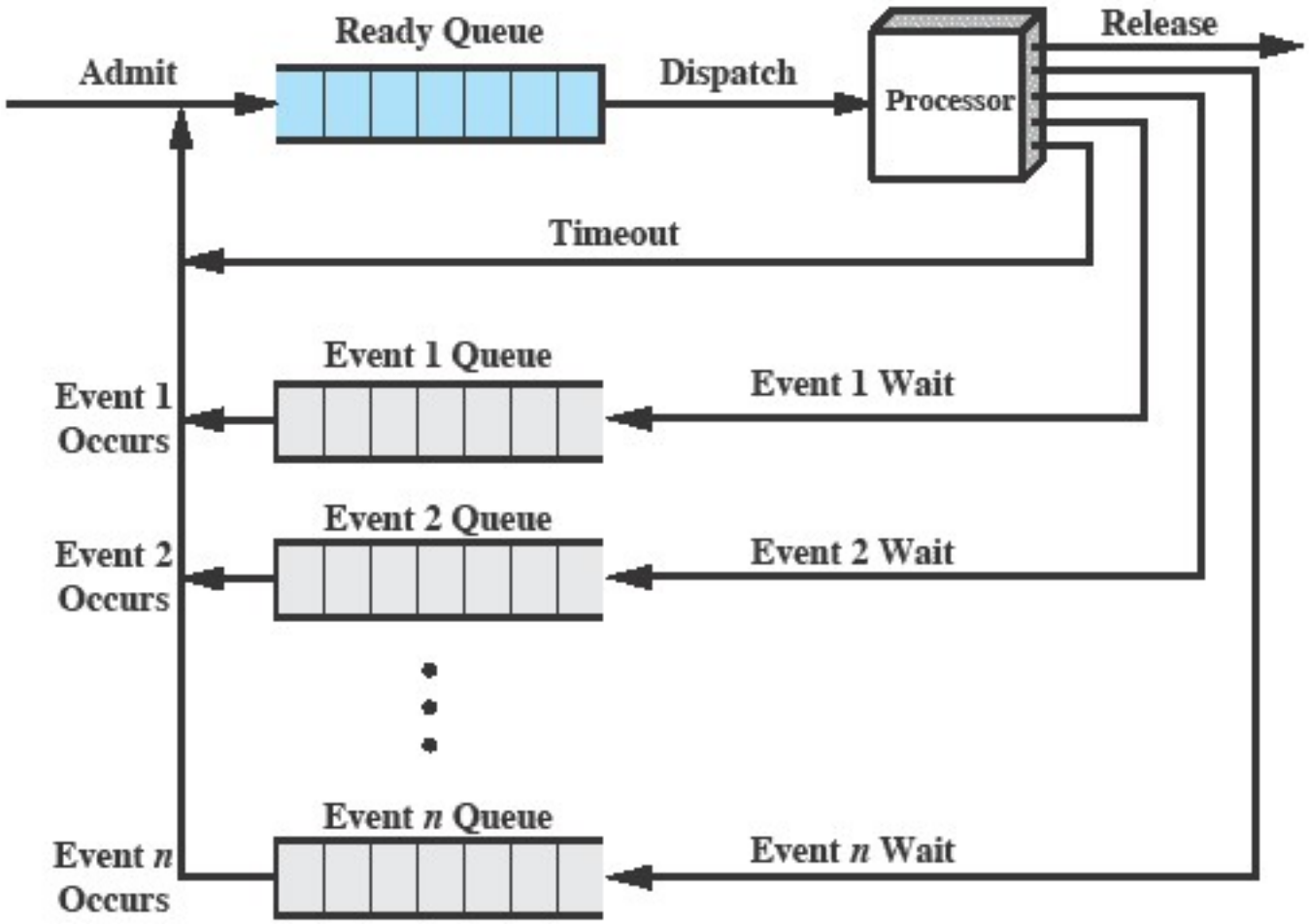
Using Two Queues



(a) Single blocked queue

But when an event occurs the dispatcher must scan the entire Blocked Queue to see which process is waiting for the event

Multiple Blocked Queues

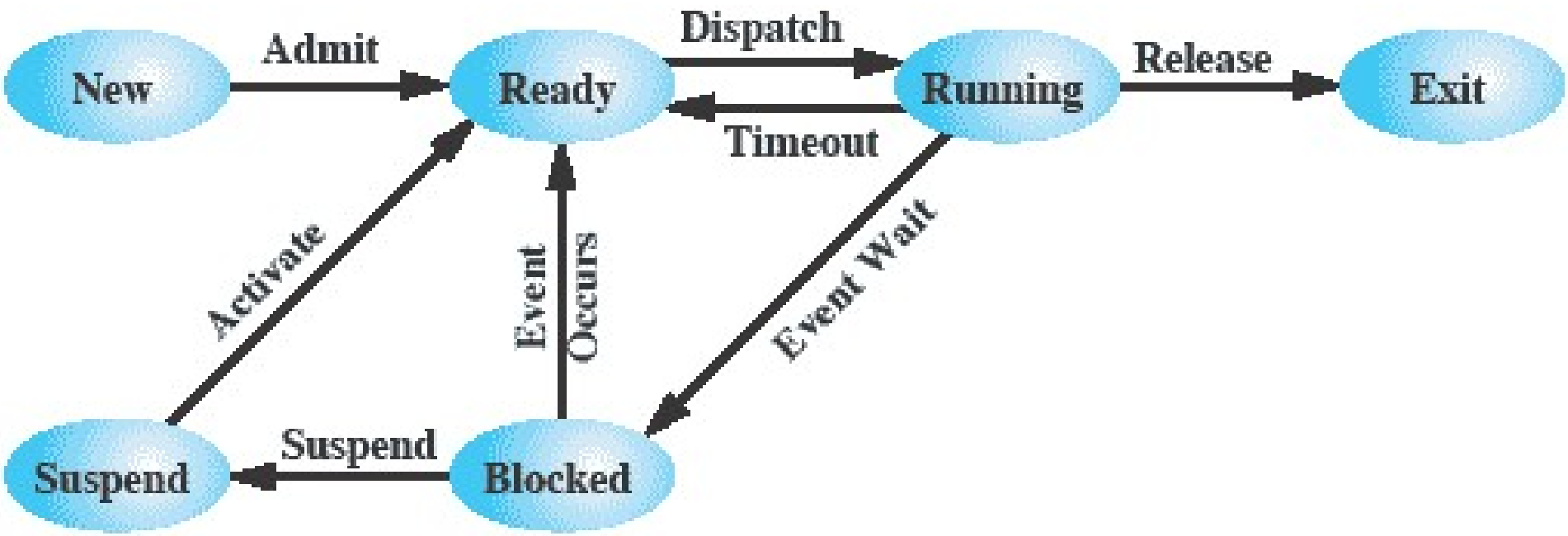


(b) Multiple blocked queues

Suspended Processes

- Processor is faster than I/O so all processes could be waiting for I/O
- Running, Ready and Blocked processes are resident in main memory
- Swap these processes to disk to free up more memory and use processor on more processes
- Blocked state becomes suspend state when swapped to disk
- Two new states
  - Blocked/Suspend
  - Ready/Suspend

One Suspend State



(a) With One Suspend State

But this only allows processes which are blocked to be swapped out.

Two Suspend States

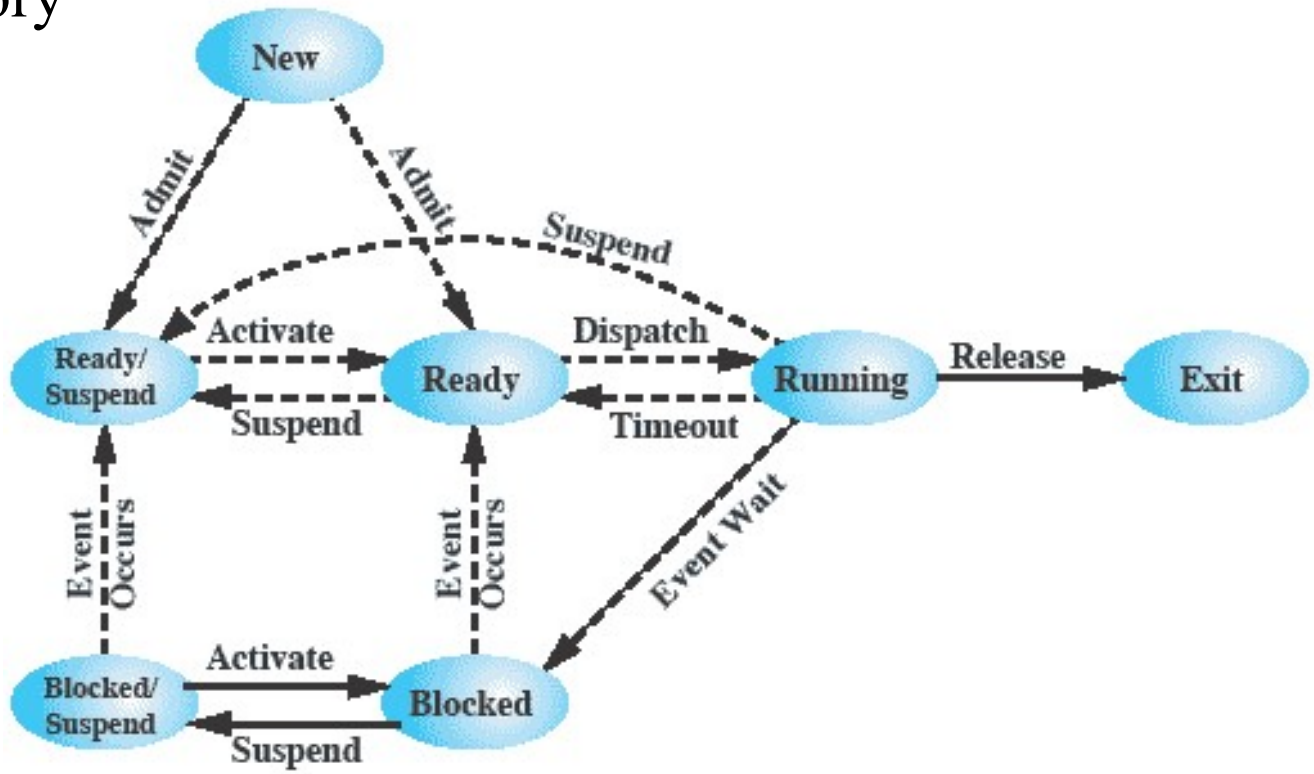
Allow all processes which are not actually running to be swapped

Ready: in main memory and available for execution

Blocked: in main memory and awaiting an event

Blocked/Suspend: in secondary memory and awaiting an event

Ready/Suspend: in secondary memory, available for execution when loaded in main memory

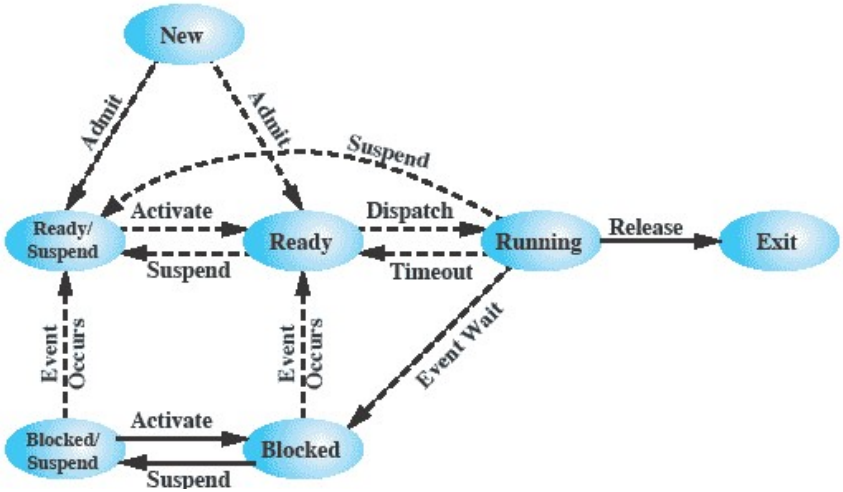


(b) With Two Suspend States

Reason for Process Suspension

Reason	Comment
Swapping	The OS needs to release sufficient main memory to bring in a process that is ready to execute OR increase the memory allocated to other Ready processes
Other OS Reason	OS suspects process of causing a problem e.g. deadlock
Interactive User Request	e.g. debugging or in connection with the use of a resource e.g. Background monitor to be turned on/off
Timing	A process may be executed periodically (e.g., an accounting or system monitoring process) and may be suspended while waiting for the next time.
Parent Process Request	A parent process may wish to suspend execution of a descendent to examine or modify the suspended process, or to coordinate the activity of various descendants.

Two Suspend States



Blocked →Blocked/Suspend make room for another process that is not blocked OR if currently running/ready process requires more main memory

Blocked/Suspend→Ready/Suspend*:* when the event occurs

Ready/Suspend→Ready*:* When there are no ready processes in main memory, or if a suspended process has a higher priority

Ready →Ready/Suspend*:* if that is the only way to free up a sufficiently large block of main memory OR suspend a lower priority ready process for a higher priority blocked process expected to unblock soon

New → Ready/Suspend and New →Ready: OS creates PCB for process and move to Ready if enough main memory or ready/Suspend otherwise

Blocked/Suspend → Blocked*:* seem to be poor design. But useful in case of high priority blocked process expected to unblock soon

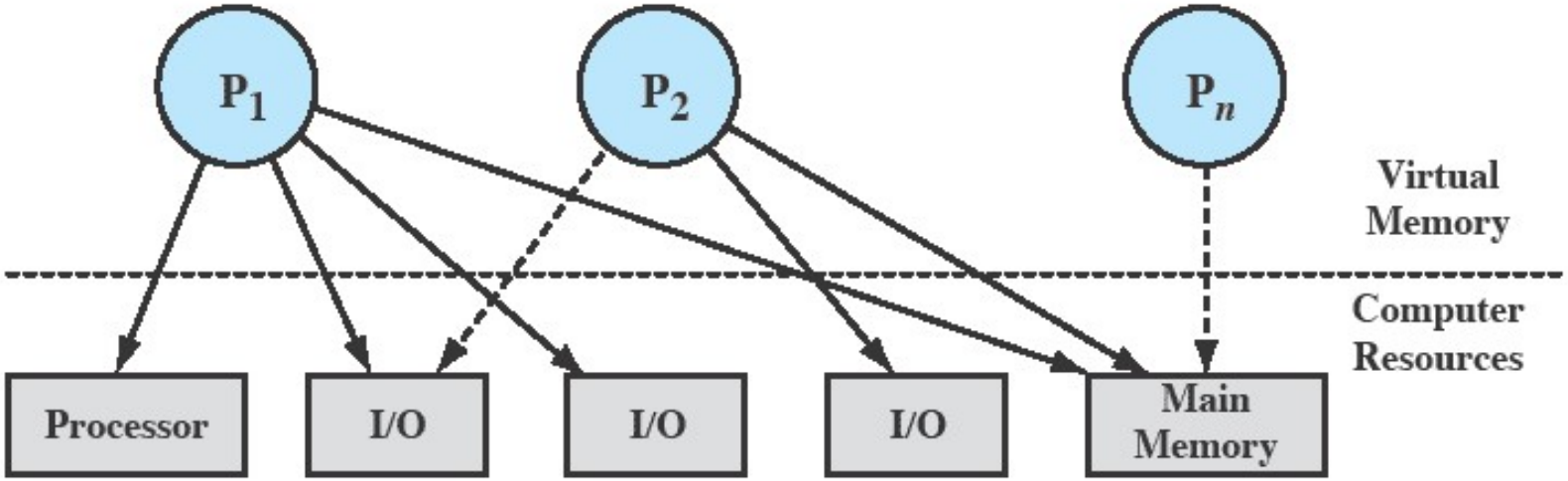
Running → Ready/Suspend*:* free some main memory.

Any State → Exit

Processes and Resources

- OS manages the use of system resources by processes
- Processes (P_1, . . ., P_n,) created and exist in memory
- Each process needs access to certain system resources
  - P_1 is running; part in main memory and has control of 2 I/O devices
  - P_2 in main memory; blocked waiting for I/O device allocated to P_1
  - P_n has been swapped out and suspended

Q: what information does OS need to control processes and manages resources for them



Operating System Control Structures

- To manage processes / resources, OS must have information about the current status of each process and resource
- Tables are constructed for each entity OS manage
  - Memory
  - I/O
  - File
  - process

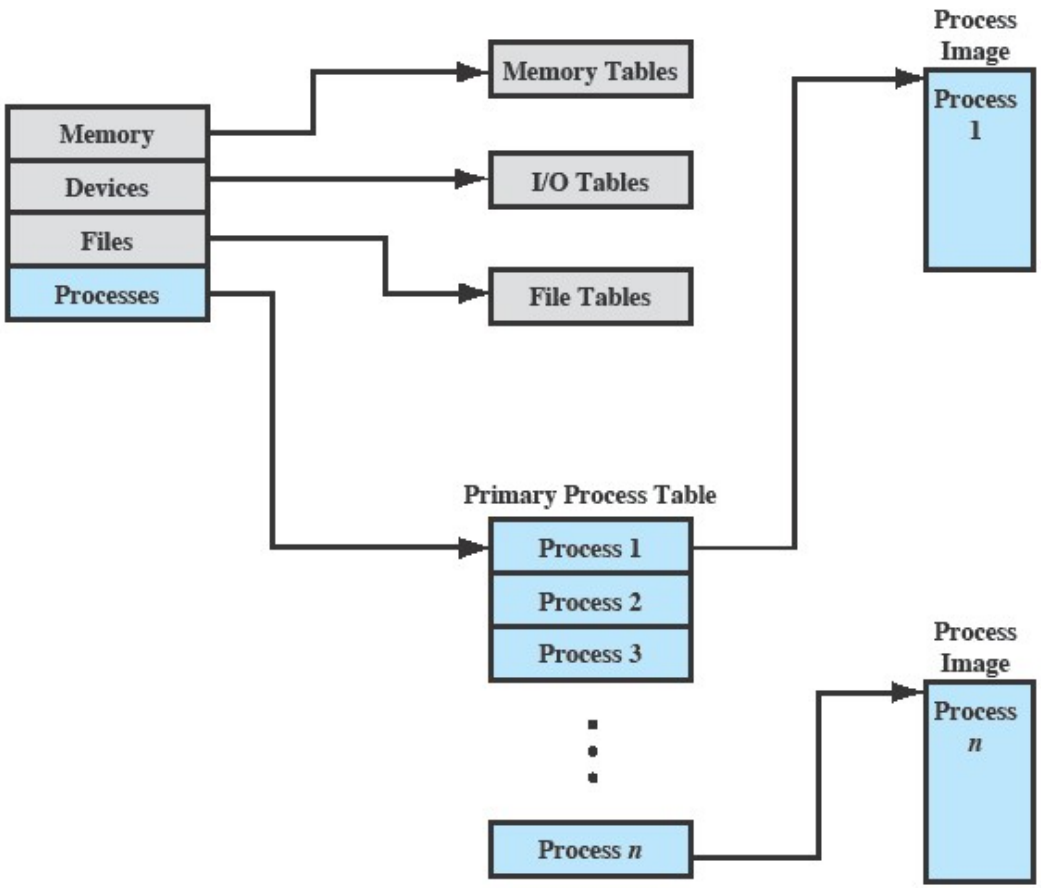


Figure 3.11 General Structure of Operating System Control Tables

Memory Tables

- Memory tables are used to keep track of both main and secondary memory
- Some of main memory is reserved for use by OS; the remainder is available for processes
- Must include this information:
  - Allocation of main memory to processes
  - Allocation of secondary memory to processes
  - Protection attributes for access to shared memory regions
  - Information needed to manage virtual memory

I/O Tables

- Used by the OS to manage the I/O devices and channels of the computer.
- The OS needs to know
  - Whether the I/O device is available or assigned
  - The status of I/O operation
  - The location in main memory being used as the source or destination of the I/O transfer

File Tables

- These tables provide information about:
  - Existence of files
  - Location on secondary memory
  - Current Status
  - other attributes
- This information is maintained by a File Management System

Process Table

- Where process is located
  - Main memory: location of each page (non-contiguous)
  - Secondary storage
- Attributes necessary for management of a process
  - Process ID
  - Process status
  - etc

Process Image

User Data

The modifiable part of the user space. May include program data, a user stack area, and programs that may be modified.

User Program

The program to be executed.

System Stack

Each process has one or more last-in-first-out (LIFO) system stacks associated with it. A stack is used to store parameters and calling addresses for procedure and system calls.

Process Control Block

Data needed by the operating system to control the process (see Table 3.5).

Process Control Block (PCB)

- PCB holds attributes of a process necessary for OS to manage the process
- Information can be grouped in three categories
  - Process identification
  - Process state information
  - Process control information
- **Process Identification**
  - Identifier of this process
    - An index in primary process table
    - Cross referenced by other tables
    - Passed on when a process communicate with another
  - Identifier of the process that created it (parent process)
  - User identifier: user responsible for the job

PCB: Processor State Information

- Consists of contents of processor registers
- User-Visible Registers
  - Typically, 16 to 32 registers that can be referenced by user programs directly
- Control and Status Registers
  - *Program counter:* contains the address of the next instruction
  - *Condition codes:* Result of the most recent arithmetic or logical operation (e.g., sign, zero, carry, equal, overflow)
  - *Status information:* interrupt enabled/disabled flags, execution mode etc.
- Stack Pointers
  - each process has one or more LIFO system stacks which is used to store parameters and calling addresses for procedure and system calls.

Program Status Word (PSW)

- All processor designs include a PSW register
- PSW contain condition codes plus other status information
- A good example of a PSW is x86 EFLAGS register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	ID	VIP	VIF	AC	VM	RF	0	NT	IOPL		OF	DF	IF	TF	SF	ZF	0	AF	0	PF	1	CF

X ID = Identification flag
X VIP = Virtual interrupt pending
X VIF = Virtual interrupt flag
X AC = Alignment check
X VM = Virtual 8086 mode
X RF = Resume flag
X NT = Nested task flag
X IOPL = I/O privilege level
S OF = Overflow flag

C DF = Direction flag
X IF = Interrupt enable flag
X TF = Trap flag
S SF = Sign flag
S ZF = Zero flag
S AF = Auxiliary carry flag
S PF = Parity flag
S CF = Carry flag

S Indicates a Status Flag
C Indicates a Control Flag
X Indicates a System Flag
Shaded bits are reserved

PCB: Processor Control Information

- Info needed by OS to Control/coordinate the active processes
- Scheduling and State Information

  needed by the OS to perform its scheduling function:

  - Process state*:* (e.g., running, ready, waiting/blocked, halted).
  - Priority: scheduling priority of the process.
  - Scheduling-related information*:* that the scheduling algorithm may need (e.g. the amount of time spent in waiting, the amount of time used during the last time it was running, etc.)
  - Event: Id of event the process is awaiting before it can run again
- Data Structuring
  - A process may be linked to other processes in a queue, ring, etc
    - Ex: all processes in a waiting state for a particular priority level may be linked in a queue
    - Ex A process may exhibit a parent-child (creator-created) relationship with another process
  - PCB contain pointers to other processes to support these structures

PCB: Processor Control Information

- Interprocess Communication info
  - Various flags, signals, and messages associated with communication between processes.
- Process Privileges
  - Processes are granted privileges in terms of the memory that may be accessed, the types of instructions that may be executed, the system utilities and services that may be used, etc.
- Memory Management
  - includes pointers to segment and/or page tables that describe the *virtual memory* assigned to this process.
- Resource Ownership and Utilization
  - Resources controlled by the process such as opened files are indicated.
  - A history of utilization of the processor or other resources may also be included (may be needed by the scheduler)

Structure of Process Image in Virtual Memory

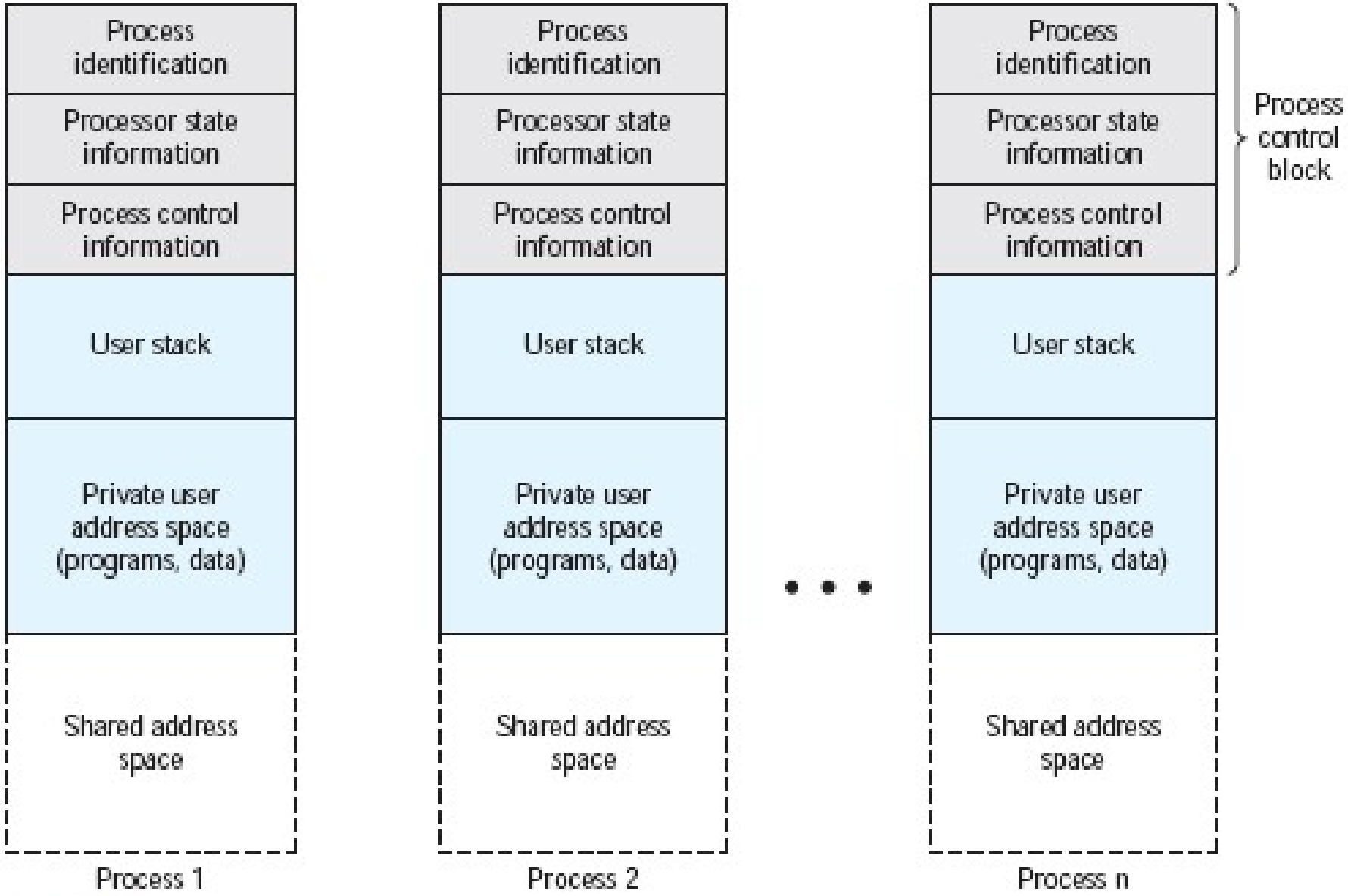


Figure 3.13 User Processes in Virtual Memory

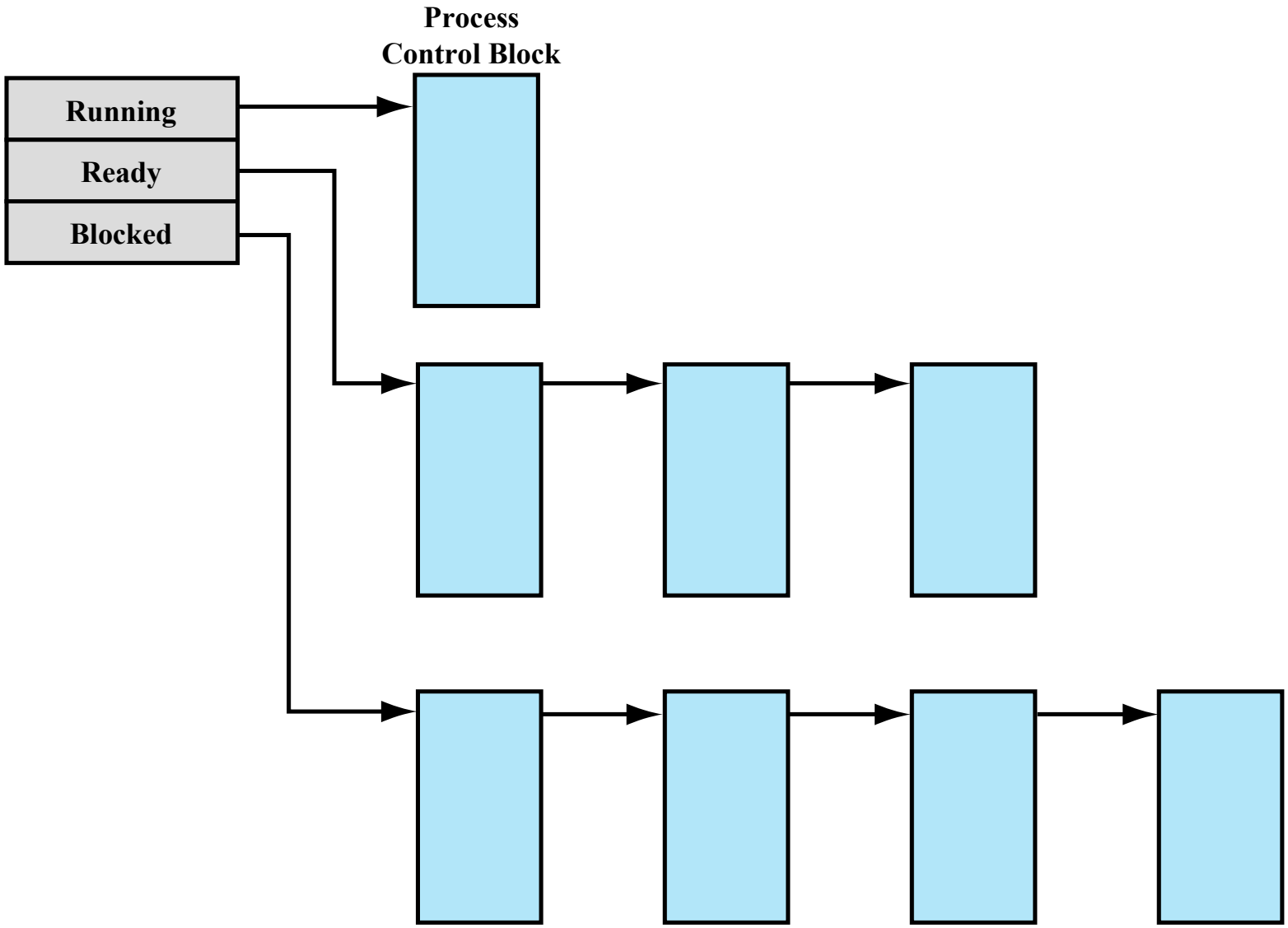


Figure 3.14 Process List Structures

Role of the Process Control Block

- The most important data structure in an OS
  - Contains all of the information about a process that is needed by the OS
  - PCBs are read and/or modified by every module in the OS (scheduling, interrupt processing, performance monitoring, etc..)
  - Defines the state of the OS
- Direct access from OS routines to PCBs raises problems
  - A bug in a single routine could damage PCBs, which could destroy the system's ability to manage the corresponding processes
  - A design change in the structure or semantics of the PCB could affect a number of modules in the OS
- One solution is to require all routines in OS to access PCBs through a handler routine, which is responsible for the protection of PCBs. But this solution raises performance issues

Modes of Execution

- Most processors support at least two execution modes
  - **User mode**
    - Less-privileged mode
    - User programs typically execute in this mode
  - **Kernel mode (system/control mode)** more-privileged mode
    - reading or altering a control register, e.g. PSW
    - primitive I/O instructions
    - Instructions that relate to memory management
    - regions of memory are accessed only in Kernel mode
- ***Q:*** How does the processor know in which mode it is to be executing?
- **A:** a flag (single bit) in PSW; changed in response to certain events e.g. Interrupt, service call to OS, etc..

Functions of OS Kernel

Process Management

- Process creation and termination
- Process scheduling and dispatching
- Process switching
- Process synchronization and support for interprocess communication
- Management of process control blocks

Memory Management

- Allocation of address space to processes
- Swapping
- Page and segment management

I/O Management

- Buffer management
- Allocation of I/O channels and devices to processes

Support Functions

- Interrupt handling
- Accounting
- Monitoring

Steps of Process Creation

- Assign a unique process identifier
  - Add entry to primary process table
- Allocate space for the process
  - Program, data, user stack, PCB
- Initialize Process Control Block
- Set up appropriate linkages
  - add new process to linked list/queue of Ready or ready/ Suspend
- Create or expand other data structures
  - Ex: maintain an accounting file for billing or performance assessment

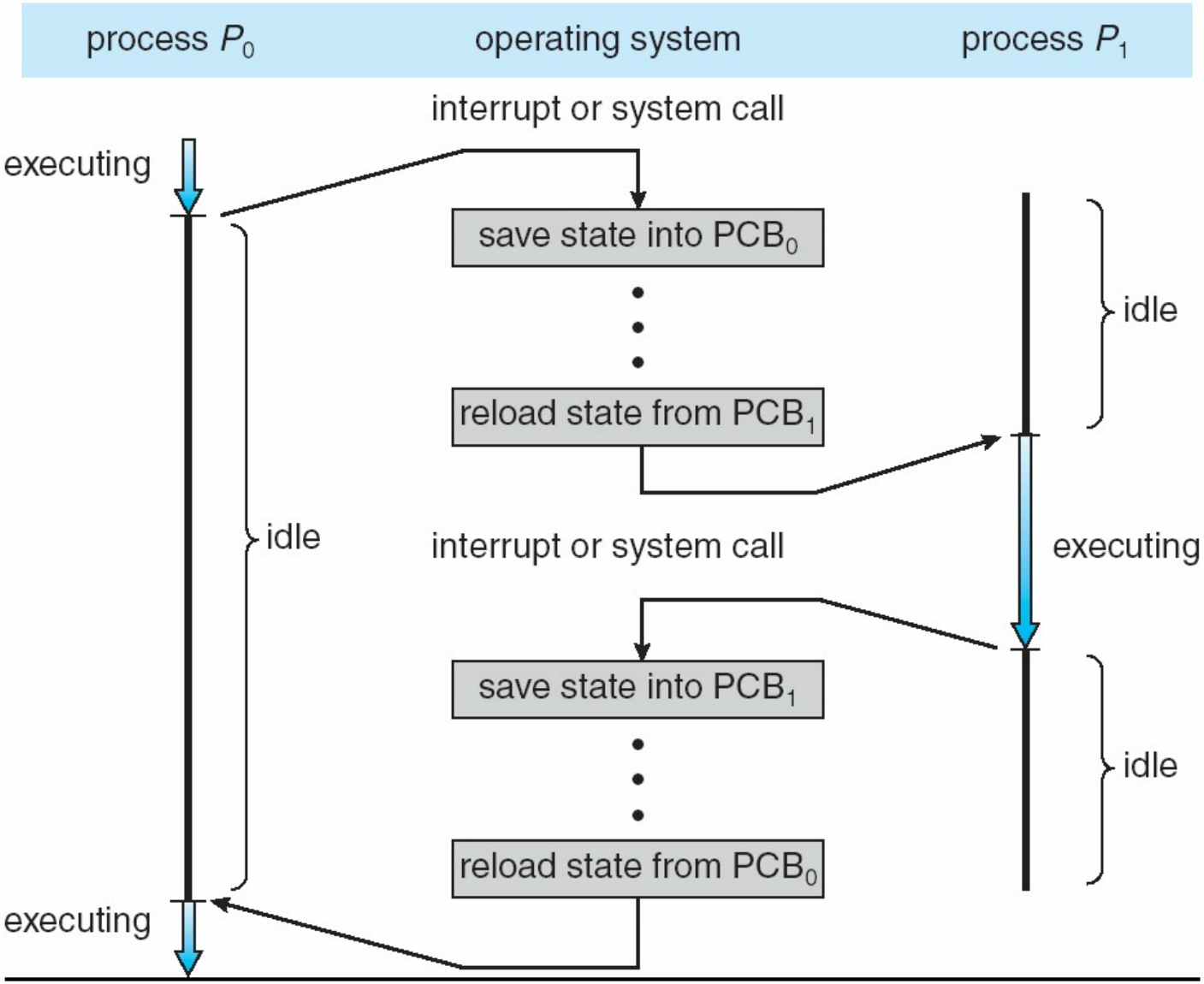
Process Switching: When to Switch a Process

- System interrupt : event external to the running process
  - Clock interrupt process has executed the maximum time slice
  - I/O interrupt e.g. I/O completion
  - Memory fault refer to address which is in virtual memory
    - OS issue I/O request to bring block into main memory
    - Current process moved to Blocked and OS switch to a Ready process
  - Control is transferred to interrupt handler, then branch to OS routine concerned with type of interrupt that occurred
- Trap: error/exception generated within the running process
  - e.g. illegal file access attempt
  - may cause process to be moved to Exit state and switch
- Supervisor call eg running process issue file open call to OS
  - switch to OS routine and user process is blocked

Change of Process State …

- Process switch involves the following steps
  1. Save context of processor including PC and other registers
  2. Update the PCB of the Running process (new state, reason for change, duration - for accounting, etc..)
  3. Move PCB to appropriate queue – Ready; Blocked; etc
  4. Select another process for execution
  5. Update the PCB of the process selected (state = Running)
  6. Update memory management data structures
  7. Restore context of the selected process (PC, registers, etc..)
- Mode switch: involves less overhead
  - Upon interrupt, interrupt handler to execute does not need full process switch. Only set PC and change mode to Kernel
  - Only Processor State Info must be saved and restored when handler is done

CPU Switch From Process to Process



Is the OS a Process?

- If the OS is just a collection of programs and if it is executed by the processor just like any other program, is the OS a process?
- If so, how is it controlled?
- Who (what) controls it?
- There are a number of design approaches
  - Non-process Kernel
  - Execution within User Processes
  - Process –Based OS

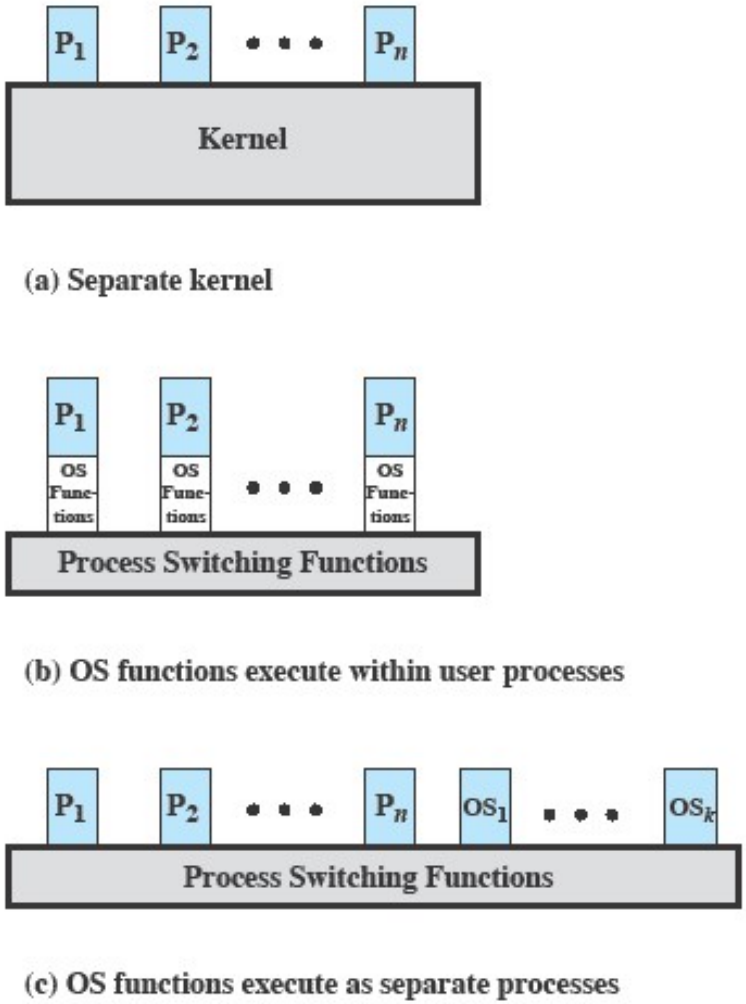
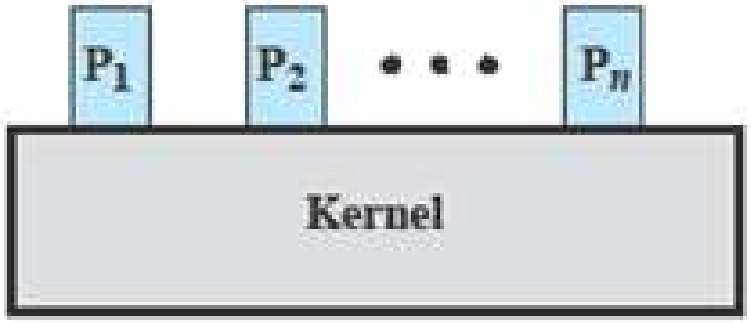


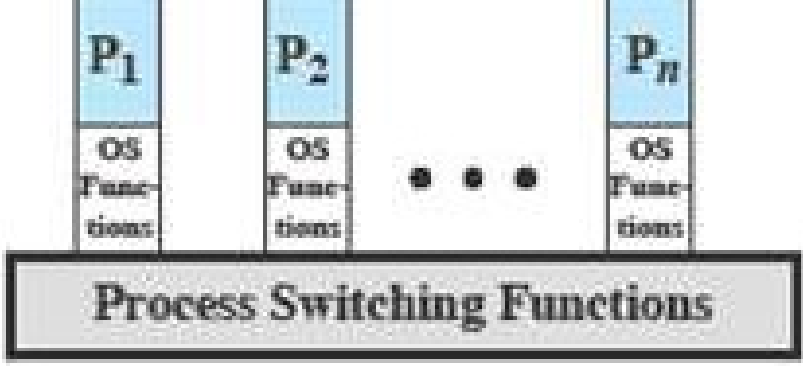
Figure 3.15 Relationship Between Operating System and User Processes

Non-process Kernel

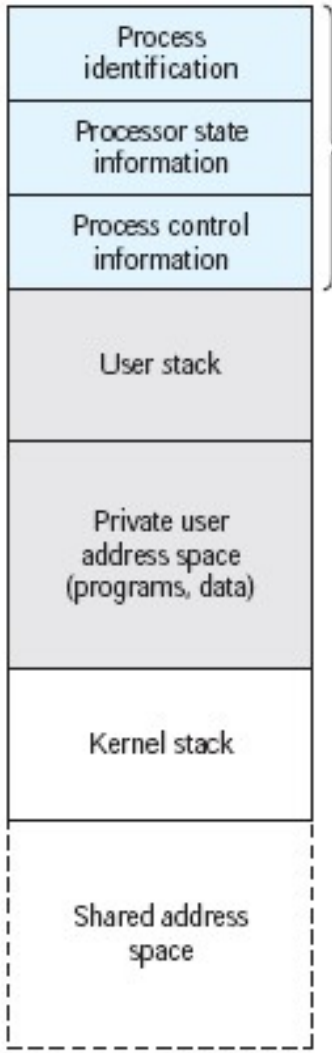


- Execute kernel outside of any process
- Common on many older operating systems
- OS has its own region of memory and own system stack
- When a process is interrupted
  - mode context is saved and control is passed to the kernel
  - Later, OS restores the context of interrupted process
- Key point
  - The concept of process is considered to apply only to user programs
  - OS code is executed as a separate entity that runs in privileged mode

Execution *Within* User Processes



- Common on small computers (PCs, WS)
- OS runs within context of a user process
- Process image structure
  - A separate kernel stack to manage calls/returns while the process is in kernel mode
  - OS code and data are in the shared address space by all processes
  - When an interrupt, trap, or supervisor call occurs, the processor is placed in kernel mode and control is passed to the OS
  - the mode context is saved and a mode switch occur to an OS routine
  - However, execution continues within the current user process. Thus, no need for process switch, just mode switch within the same process
  - If OS upon completion, determines that current process should resume, then a mode switch resumes the interrupted process.



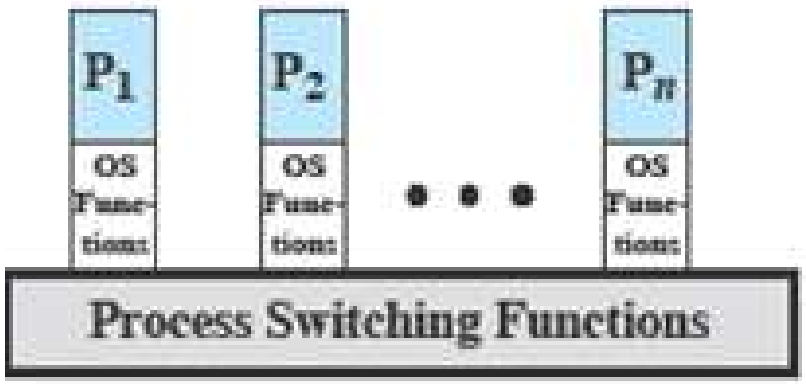
- key advantages: A user program has been interrupted to employ some OS routine, and then resumed, without incurring the penalty of two process switches.

Process-based OS

- Implement the OS as a collection of system processes
- As before, the software of the kernel executes in a kernel mode
- But, major kernel functions are organized as separate processes
- There may be a small amount of process switching code
- Advantages.
  - imposes a program design discipline that encourages the use of a modular OS with minimal, clean interfaces between the modules.
  - noncritical OS functions are implemented as separate processes
  - useful in a multiprocessor or multicomputer environment, in which some OS services can be shipped out to dedicated processors, improving performance.

Unix SVR4 System V Release 4

- Most of the OS executes within the environment of a user process
- System Processes
  - run in Kernel mode
  - Execute OS code e.g. Memory allocation, process swapping
- User Processes operates in
  - User mode to execute user programs and utilities
  - Kernel mode to execute instructions belonging to the kernel
  - A user process enters kernel mode
    - by issuing a system call
    - when exception is generated
    - when an interrupt occurs

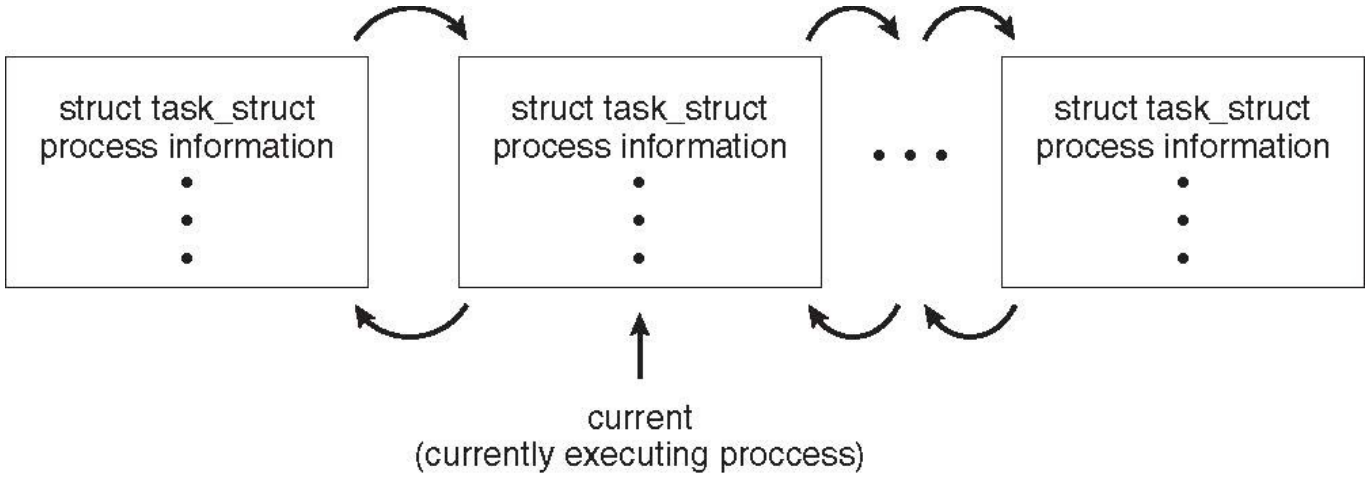


Process Representation in Linux

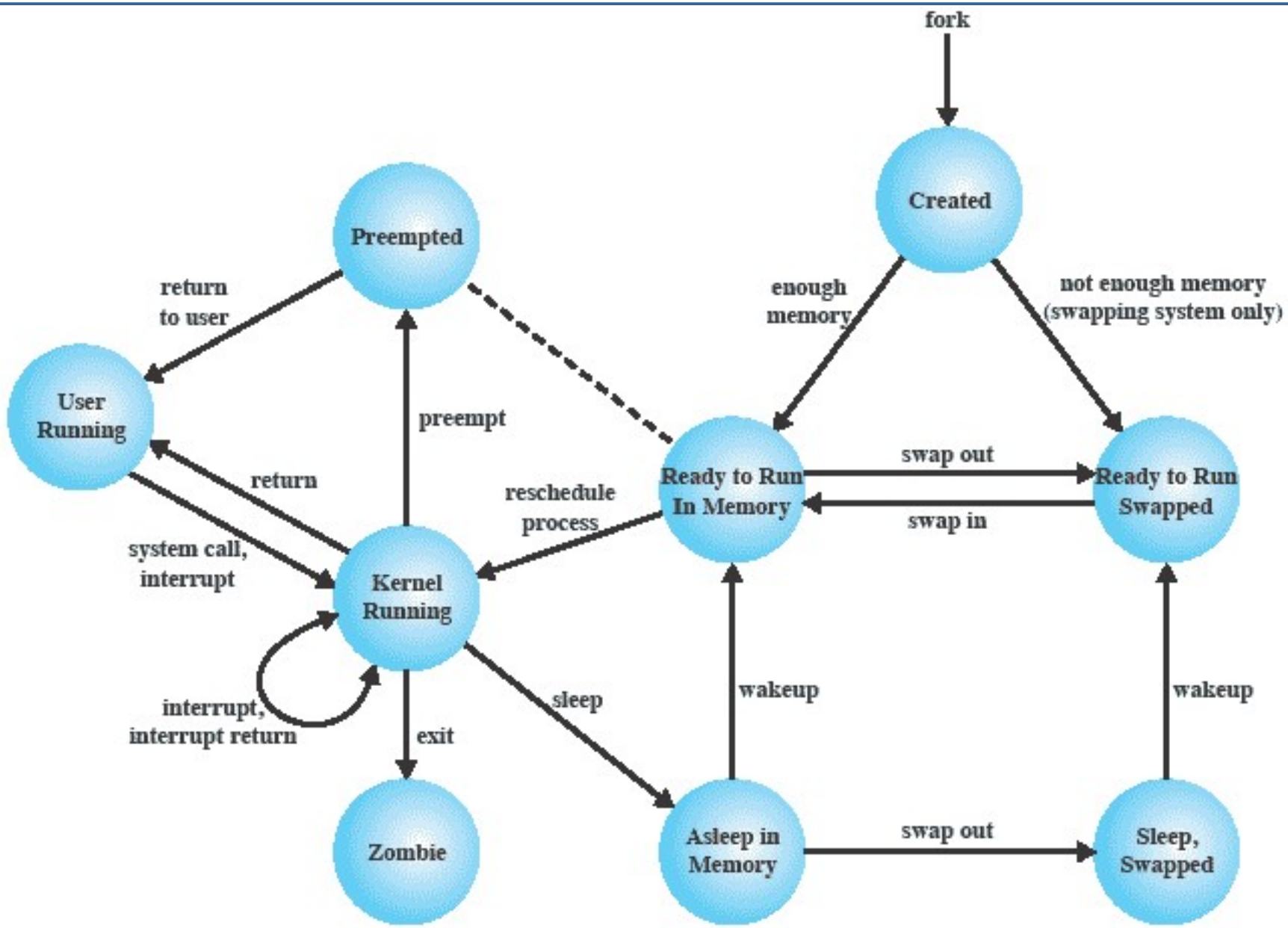
PCB is Represented by the C structure **`task_struct`**

```
pid t_pid; /* process identifier */
long state; /* state of the process */
unsigned int time_slice /* scheduling information */
struct task_struct *parent; /* this process's parent */
struct list_head children; /* this process's children */
struct files_struct *files; /* list of open files */
struct mm_struct *mm; /* address space of this process */
```

- all active processes are represented using a doubly linked list of **`task_struct`**
- The kernel maintains a pointer **`current`** to the process currently executing



UNIX Process State Transition Diagram



UNIX Process States

User Running	Executing in user mode.
Kernel Running	Executing in kernel mode.
Ready to Run, in Memory	Ready to run as soon as the kernel schedules it.
Asleep in Memory	Unable to execute until an event occurs; process is in main memory (a blocked state).
Ready to Run, Swapped	Process is ready to run, but the swapper must swap the process into main memory before the kernel can schedule it to execute.
Sleeping, Swapped	The process is awaiting an event and has been swapped to secondary storage (a blocked state).
Preempted	Process is returning from kernel to user mode, but the kernel preempts it and does a process switch to schedule another process.
Created	Process is newly created and not yet ready to run.
Zombie	Process no longer exists, but it leaves a record for its parent process to collect.

UNIX Process Image

User-Level Context	
Process Text	Executable machine instructions of the program
Process Data	Data accessible by the program of this process
User Stack	Contains the arguments, local variables, and pointers for functions executing in user mode
Shared Memory	Memory shared with other processes, used for interprocess communication
Register Context	
Program Counter	Address of next instruction to be executed; may be in kernel or user memory space of this process
Processor Status Register	Contains the hardware status at the time of preemption; contents and format are hardware dependent
Stack Pointer	Points to the top of the kernel or user stack, depending on the mode of operation at the time or preemption
General-Purpose Registers	Hardware dependent
System-Level Context	
Process Table Entry	Defines state of a process; this information is always accessible to the operating system
U (user) Area	Process control information that needs to be accessed only in the context of the process
Per Process Region Table	Defines the mapping from virtual to physical addresses; also contains a permission field that indicates the type of access allowed the process: read-only, read-write, or read-execute
Kernel Stack	Contains the stack frame of kernel procedures as the process executes in kernel mode

Process Creation in Unix

- Process creation is by means of the system call fork()
- This causes the OS, in Kernel Mode, to:
  1. Allocate a slot in the process table for the new process.
  2. Assign a unique process ID to the child process.
  3. Copy of process image of the parent, with the exception of any shared memory.
  4. Increment the counters for any files owned by the parent, to reflect that an additional process now also owns those files.
  5. Assign the child process to the Ready to Run state.
  6. Returns the ID number of the child to the parent process, and a 0 value to the child process.

After Creation

- After creating the process the Kernel can do one of the following, as part of the dispatcher routine:
  - Stay in the parent process. Control returns to user mode at the point of the fork call of the parent.
  - Transfer control to the child process. The child process begins executing at the same point in the code as the parent, namely at the return from the fork call.
  - Transfer control to another process. Both parent and child are left in the Ready to Run state.

UNIX Process Table Entry

Process status	Current state of process.
Pointers	To U area and process memory area (text, data, stack).
Process size	Enables the operating system to know how much space to allocate the process.
User identifiers	The **real user ID** identifies the user who is responsible for the running process. The **effective user ID** may be used by a process to gain temporary privileges associated with a particular program; while that program is being executed as part of the process, the process operates with the effective user ID.
Process identifiers	ID of this process; ID of parent process. These are set up when the process enters the Created state during the fork system call.
Event descriptor	Valid when a process is in a sleeping state; when the event occurs, the process is transferred to a ready-to-run state.
Priority	Used for process scheduling.
Signal	Enumerates signals sent to a process but not yet handled.
Timers	Include process execution time, kernel resource utilization, and user-set timer used to send alarm signal to a process.
P_link	Pointer to the next link in the ready queue (valid if process is ready to execute).
Memory status	Indicates whether process image is in main memory or swapped out. If it is in memory, this field also indicates whether it may be swapped out or is temporarily locked into main memory.

UNIX User Area

Process table pointer	Indicates entry that corresponds to the U area.
User identifiers	Real and effective user IDs. Used to determine user privileges.
Timers	Record time that the process (and its descendants) spent executing in user mode and in kernel mode.
Signal-handler array	For each type of signal defined in the system, indicates how the process will react to receipt of that signal (exit, ignore, execute specified user function).
Control terminal	Indicates login terminal for this process, if one exists.
Error field	Records errors encountered during a system call.
Return value	Contains the result of system calls.
I/O parameters	Describe the amount of data to transfer, the address of the source (or target) data array in user space, and file offsets for I/O.
File parameters	Current directory and current root describe the file system environment of the process.
User file descriptor table	Records the files the process has opened.
Limit fields	Restrict the size of the process and the size of a file it can write.
Permission modes fields	Mask mode settings on files the process creates.